RAT (RACE AGAINST TIME)

Digital Design Document

SRM Team Robocon

1. ROBOT DESCRIPTION

RAT (Race Against Time) is a small two wheel differential drive robot that solves a maze it has never seen. The layout may change between attempts, routes may be blocked and new obstacles may appear. RAT must therefore sense its surroundings, decide its moves without human help, carry up to 3 kg and get faster on repeated attempts.

- **Chassis:** 270 × 270 mm, two plywood platforms;
- **Drive:** two traction wheels (94mm outer, 71mm inner) and one castor
- **Actuators:** 2 × PG36 24 V geared DC motors with encoders, driven by a Cytron MDDS30
- **Sensors:** 3 × HC SR04 ultrasonic, 2 motor encoders, MPU 6050 IMU
- **Controller and power:** ESP32; 6S LiPo (22.2 V) with LM2596 buck converter to 5 V
- **Maze:** 300×300 cm, 6×6 cells of 50cm

RAT explores in Run 1 with left hand wall following and records every move. In Run 2 it replays a shortened path at higher speed.

RAT combines three kinds of sensing, which is multi sensor integration. The ultrasonic sensors tell RAT where the walls are, the wheel encoders tell it how far it has moved, and the IMU tells it how much it has turned. No single sensor is enough on its own: an ultrasonic reading says nothing about distance travelled, an encoder only counts wheel rotation, and the IMU cannot see walls. Together they let RAT choose a move from the walls it sees, carry it out accurately and check the result. Because the maze may change between attempts, RAT never assumes a stored layout; it trusts only what it senses at each cell.

- **RAT Image:**

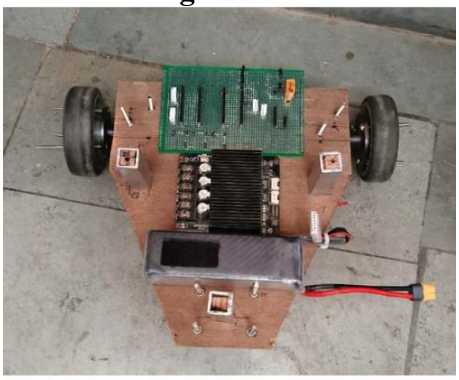


Figure 1: Top View



Figure 2: Side View

2.MECHANICAL DESIGN

2.1 Structure and Fabrication

The base is made of two plywood platforms cut with a jigsaw. The lower platform carries the drive motors underneath, each connected to its wheel through motor mounts bolted to the base. An elevated platform above it, held up by aluminium channels, leaves room for the electronics, and a top platform holds the load. The channels are cut with an angle grinder, reinforced with wood and bolted in place.

The traction wheels are basic rubber wheels, 94 mm outer diameter and 71 mm inner diameter, joined to the motor shafts with flange couplers (PCD 47.5 mm). One castor wheel supports the chassis and helps the robot change direction. Omni wheels were not chosen because they slip, which would spoil the encoder measurement.

2.2 Design Calculations

- **Speed:**
  rated 430 rpm = 7.17 rev/s (ω = 45.0 rad/s)
  power 0.716 × 45.0 = 32.2 W per motor
- **Wheel and top speed:**
  circumference π × 94 = 295.3 mm; theoretical top speed π × 0.094 × 7.17 = 2.12 m/s. Run 1 is driven much slower
- **Pulling force:**
  F = 1.43 ÷ 0.047 m = 30.4 N
- **Acceleration:**
  a = F ÷ m. With an assumed 2 kg robot plus 3 kg payload (5 kg), a ≈ 5.8 m/s². With an approx friction coefficient of 0.7, grip allows about 6.9 m/s², so the wheels should not slip.
- **Cell distance:**
  500 ÷ 295.3 = 1.69 wheel turns per cell.
  Ticks per cell = 1.69 × N, where N (encoder counts per turn, PPR × gear ratio) .
  Distance per tick = 295.3 ÷ N mm
- **Motor Torque:**
  7.3 kgf.cm × 9.81 ÷ 100 = 0.716 N.m per motor;
  1.43 N.m for both motor
- **Calibration:**
  push RAT one cell by hand, read the counts and use the measured value
- **Turning clearance:**
  chassis diagonal 270 × √2 = 382 mm, less than the ~470 mm clear cell width, so RAT can turn in place (axle at chassis centre)

From these data one can conclude that there is margin in the drive system. Two motors combined have enough force to pull 29.2 N, 60% of the total mass of the supposed 3 kg of the robot + payload (3 x 9.81 = 29.4 N). Acceleration of 5.8 m/s² is less than the limit value provided by grip of 6.9 m/s² (0.7 x 9.81), hence wheels will not skid. Since the castor takes part of the robot's load, this limit is smaller, therefore the test needs to be carried out again with the actual mass of the robot. The maximum speed of 2.21 m/s is an absolute theoretical value for this speed of motors. In cells of 50 cm, RAT requires precise stopping and turning more than any speed; that is why Run 1 was run much slower.

2.3 CAD Models

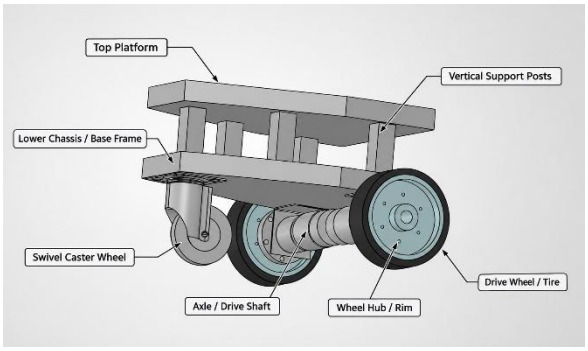


Figure 1: CAD isometric view of the complete assembly

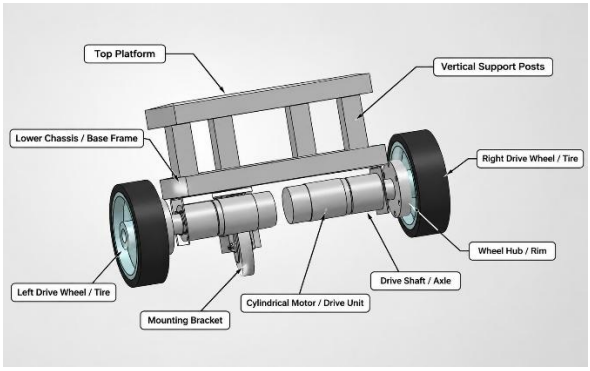


Figure 2: CAD underside view showing motor and drive wheel layout

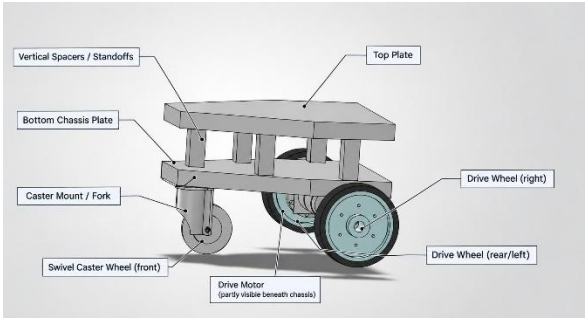


Figure 3: CAD alternate isometric view showing castor and drive wheel

3. CIRCUIT DESIGN AND PCB

3.1 Architecture and Power

The ESP32 is the central controller. It stores the control logic in flash memory, processes sensor inputs with its processor and RAM, and decides when and how much signal to send to each output pin. A 6S LiPo (22.2 V nominal, 25.2 V full) powers the 24 V motors through the Cytron MDDS30 (SmartDriveDuo 30), a dual channel driver that runs both motors from one unit. An LM2596 buck converter steps the battery down to 5 V for the ESP32 and sensors. Its output is set to 5 V with a multimeter before the ESP32 is connected. All grounds are joined to one common GND.

The robot works at two power levels. The 24 V motors draw high current, so they are fed from the battery through the MDDS30, which turns the PWM and DIR signals from the ESP32 into motor power. The ESP32, sensors and encoders need only 5 V, so the LM2596 buck converter steps the battery voltage down. A buck converter wastes much less energy than a linear regulator, which would burn the extra voltage as heat. Joining all grounds gives every signal the same zero volt reference. The MPU6050 talks to the ESP32 over I2C, a two wire bus in which SDA (GPIO 21) carries the data and SCL (GPIO 22) carries the clock.

3.2 Components, Sensors and Connections

- **Ultrasonic (×3, left, front, right):** measure wall distance. They judge junction openings to choose the turn direction and give forward collision detection
- **Encoders (×2, behind each motor):** give RPM, distance travelled and position. Power consumption is estimated from motor current (Section 3.4)
- **MPU 6050 IMU (I2C, SDA GPIO 21, SCL GPIO 22):** accelerometer gives linear acceleration, gyroscope gives angular velocity. Yaw gives 90° and 180° turn angles and detects drift for realignment
- **MDDS30 driver:** per channel, PWM sets speed and DIR sets direction
- **Pins:** all pin numbers are kept in one file. Encoder and echo inputs are planned on GPIO 34 to 39 (input-only). GPIO 12 is avoided because it can stop the ESP32 from booting. Final pin list TBC
- **Echo level:** HC SR04 echo is 5 V, so a 1 kΩ / 2 kΩ divider gives 5 × 2 ÷ 3 = 3.33 V

3.3 Power Budget and Battery Sizing

Worst case at 22.2 V, with the motor rated current of 4 A as a safe upper bound:

- Two motors:
  2 × 4 A × 22.2 V = 177.6 W
- 5 V logic:
  0.329 A (ESP32 0.240 A, sensors 0.045 A, MPU-6050 0.004 A, encoders 0.040 A TBC), 1.65 W, or 1.94 W from the battery at 85% efficiency
- **Total:**
  179.5 W, about 8.1 A.

A 10 minute run needs 8.1 × 10 ÷ 60 = 1.35 Ah, or 1.69 Ah at 80% usable capacity (about 37.7 Wh). The discharge rate is about 4.8 C, so a pack rated 5 C or more is chosen.

Pack chosen: 6S 6800 mAh pack with a 40C discharge rating. The 25.2 V full voltage is below the LM2596 input limit of about 32 V .

3.4 Schematic and PCB

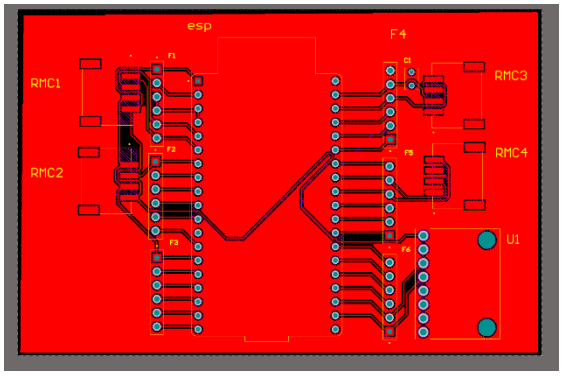


Figure 4: PCB Layout

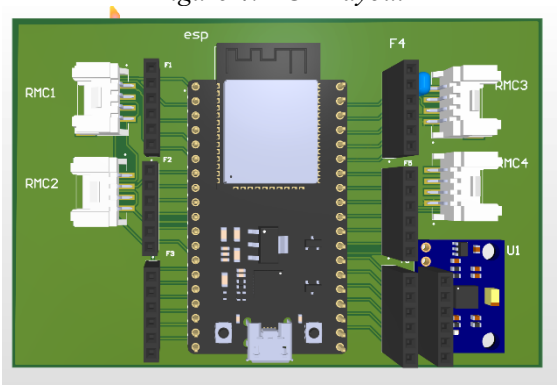


Figure 5: 3D PCB Render

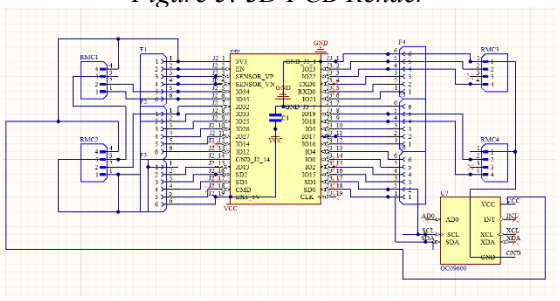


Figure 6: Circuit Schematic Diagram

4. NAVIGATION: LEFT HAND ALGORITHM

4.1 Sensing

Each ultrasonic sensor sends a 40 kHz pulse and times the echo, so distance = 343 m/s × echo time ÷ 2 (about 20 °C). A reading below the wall threshold means "wall", otherwise the side is "open". The threshold is set in the real maze. Sensors are read one at a time, 60 ms apart, to avoid crosstalk. A wall 10 cm away echoes in 0.58 ms, the 300 cm goal in 17.5 ms, and a worst case decision (three sensors, three readings each) takes 540 ms.

4.2 Left Hand Rule

At every junction RAT reads left, front and right, and takes the first open direction:

1) Left open: turn left, move one cell
2) Front open: move straight one cell
3) Right open: turn right, move one cell
4) All blocked: turn 180°, move one cell

The rule needs no full map. It can loop forever in a maze with islands, so every layout is designed loop free with the goal on the outer wall.

The rule works because of how a loop free maze is built. With no loops and no islands, every wall is joined to the outer wall, so all the walls form one connected wall. If RAT keeps its left side along that wall, it travels along the whole wall and cannot skip a branch. Since the goal is a gap in the outer wall, the wall RAT is following eventually leads it to the gap. The route can be long, because RAT also enters dead ends and comes back out, and this is why Run 2 needs a shorter path. The rule fails when the goal is inside an island, because the wall around it is not joined to the outer wall.

4.3 Moving, Turning and Goal Detection

To move one cell, both encoders are reset and both motors run until the average count reaches the ticks per cell, then stop. While driving, the side sensors correct drift toward a wall. To turn, the motors run in opposite directions until the IMU yaw changes by 90° (180° at a dead end). The goal is a gap in the outer wall. When the front sensor reads 300 cm or more, RAT stops. This needs at least 300 cm of open space beyond the gap, to be tested in the real room.

4.4 Path Recording and Run 2

In Run 1 every move (L, R, S, U) is stored in an array in RAM. 100 moves take only 100 bytes. After the goal, dead ends are removed with three rules: **L U L → S**, **L U S → R** and **S U L → R**.

The three rules come from adding up the turns. Count a left turn as −90°, a right turn as +90°, a U-turn as 180° and straight as 0°. Then L U L = −90 + 180 − 90 = 0°, which is the same as going straight, so it becomes S. L U S = −90 + 180 + 0 = 90°, a right turn, so it becomes R. S U L = 0 + 180 − 90 = 90°, so it also becomes R. In each case the U-turn marks a dead end, and removing the detour keeps the same final heading while cutting the distance travelled.

For example, S L U L S R becomes S S S R. In Run 2 RAT replays the short path at higher speed. If a wall appears where open space was expected, it returns to wall following and records a new path.

4.5 Edge Cases and Fixes

- **Overshoot or undershoot:** slow down near the target; the front sensor acts as a stop
- **Ultrasonic misread:** take three readings and use the middle one
- **Stuck or wheel slip:** stop the motors if encoder counts don't change for 1 s
- **Wall moved before Run 2:** fall back to wall following
- **Low battery:** speed drops; recalibrate with a charged battery
- **Gyro drift:** keep RAT still while the IMU calibrates at start up

4.6 Software Structure

All code is written in the Arduino IDE and stored in ESP32 flash. **Motor** (PWM and DIR, private pins), **Encoder** (wheel ticks), **Ultrasonic** (one distance), **IMU** (yaw angle) and **Robot** (cell move, 90° and 180° turns) are the hardware layer. **Solver** is the base class that chooses a move, **WallFollower** implements the left hand rule, and **PathRecorder** stores and shortens the path. The solver only calls Robot and never touches hardware. Testing of thresholds, ticks per cell and IMU turns is pending.

4.7 Algorithm Flowchart

Run 1 steps: start, initialise parts, check for goal, read three sensors, apply the priority rule, turn, move one cell, repeat, stop at goal.

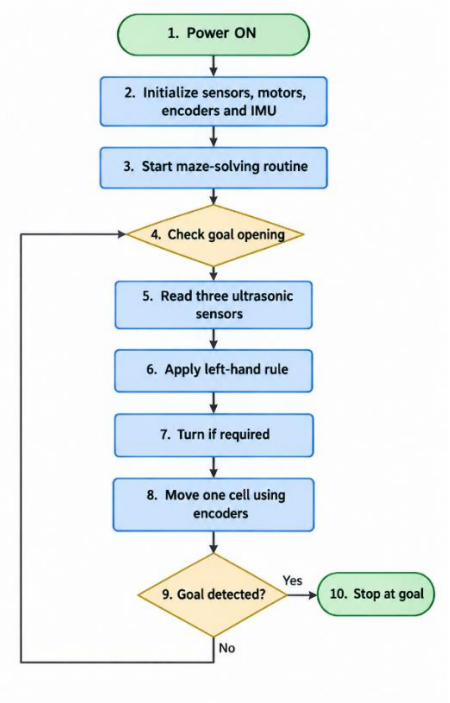


Figure 7: Algorithm Flowchart

5. SYSTEM INTEGRATION

RAT works as one system built from three parts. The mechanical structure holds the motors, wheels, electronics and load. The electronics supply power and carry the 2 sensor signals. The code decides each move. The battery powers the motors through the MDDS30 driver and the ESP32 and sensors through the LM2596 buck converter. At every cell, the three ultrasonic sensors send wall data to the ESP32, and the left-hand rule picks the next move. The Robot class then drives the motors to carry it out. The encoders measure the distance moved and the IMU measures the turn angle, and both report back until the cell or turn is complete. This repeats until the front sensor sees the open goal gap and RAT stops.